# Oracle SQL questions

1. **Maximum salary**
   SELECT MAX(salary) FROM employees
2. **Second Maximum Salary**
   SELECT MAX(salary) FROM employees WHERE salary <> (SELECT MAX(salary) FROM employees)
3. **Nth Maximum Salary**
   SELECT * FROM (SELECT salary,dense_rank() over(order by salary desc) dr FROM employees) WHERE dr=3
4. **Maximum Salary department wise**
   SELECT department_id,max(salary) FROM employees GROUP BY department_id ORDER BY department_id
5. **Nth Maximum Salary Department Wise**
   SELECT * FROM (SELECT department_id,salary,dense_rank() over(partition by department_id order by salary desc) dr FROM employees) WHERE dr=2
6. **Employee Name , Manager Name**
   SELECT e.employee_id,e.first_name,m.employee_id mgr_id,m.first_name mgr_name
   FROM employees e,employees m
   WHERE e.manager_id=m.employee_id
7. **To Find Duplicates**
   SELECT first_name,count(*) FROM employees GROUP BY first_name HAVING count(*) > 1

   SELECT * FROM employees WHERE rowid IN (SELECT max(rowid) FROM employees GROUP BY rowid)

8. **To Delete Duplicates**
   DELETE FROM employees WHERE first_name NOT IN (SELECT first_name,count(*) FROM employees GROUP BY first_name HAVING count(*) > 1)

   DELETE FROM employees WHERE rowid NOT IN (SELECT max(rowid) FROM employees GROUP BY rowid)
9. **Execution Order of Query**

| ORDER | CLAUSE | FUNCTION |
|---|---|---|
| 1 | from | Choose and join tables to get base data. |
| 2 | where | Filters the base data. |
| 3 | group by | Aggregates the base data. |
| 4 | having | Filters the aggregated data. |
| 5 | select | Returns the final data. |
| 6 | order by | Sorts the final data. |
| 7 | limit | Limits the returned data to a row count. |

10. **Null Functions**

- NVL(expr1, expr2) : In SQL, NVL() converts a null value to an actual value.
  NVL(commission_pct, 0)
- NVL2(expr1, expr2, expr3) : The NVL2 function examines the first expression. If the first expression is not null, then the NVL2 function returns the second expression
  NVL2(commission_pct, 'SAL+COMM', 'SAL')
- COALESCE() : The COALESCE() function examines the first expression, if the first expression is not null, it returns that expression
  COALESCE(commission_pct, salary, 10)
- NULLIF() : The NULLIF function compares two expressions. If they are equal, the function returns null. If they are not equal, the function returns the first expression.
  NULLIF(LENGTH(first_name),LENGTH(last_name))

**11. Data types in SQL**

- Number
- Char
- Varchar2
- Date
- Long
- CLOB - Character Large Object
- BLOB-Binary Large Object

**12. What is DDL**

DDL (Data Definition Language) is a language used by a database management system (like Oracle) that allows users to define the database and specify data types, structures and constraints on the data.
Examples DDL statements are:

- CREATE TABLE
- ALTER
- RENAME
- DROP
- TRUNCATE

**13. What is DML**

Data manipulation language (DML) statements add, change, and delete Oracle Database table data.

- INSERT
- UPDATE
- DELETE
- MERGE

**14. What is TCL**

Transaction Control Language commands are used to manage transactions in the database. These are used to manage the changes made by DML-statements.

- COMMIT
- ROLLBACK
- SAVEPOINT

**15. Types of Joins**

- Equijoin/Inner Join(Matched Records From both Tables)
- Self Joins(Joins the Table Within Itself)
- Cross Join / Cartesian Products
- Outer Joins
- Left Outer Join(Matched Records From both Table, Unmatched Record from left side Table )
- Right Outer Join(Matched Record From both Table, Unmatched Record From Right side Table)
- Full Outer Join(Matched and unmatched Record From both Table)

**16. Constraints in SQL**

- Primary key(It cannot be duplicate and Null, one table only one primary key will be used)
- Foreign Key(It allow duplicates and Null, multiple foreign key can be used)
- Not Null
- Checkz
- Unique(It is an unique Integrity, won't allow duplicates, allows null values)

**17. What is Subquery**

A query within a query is known as subquery

**18. Types of Subqueries**

- Single row subquery
- Multiple column subquery
  If Condition have more than one column than it is multiple column subquery
- Inline view
  If you use select statement in from clause than it is inline view
- Scalar Subquery
  If you use select statement inside select statement then it is scalar subquery
- Corelated Subquery
  Outer query refers inner query value or inner query refers outer query values then it is correlated subquery
- Nested Subquery
  Using select statement in where clause then it is Nested Subqueries
- A query with another query is called as Subqueries

**Query Example**

**Single row Subqueries**

select * from employees where salary =(select max(salary) from employees)

- **multiple column Subqueries**
  select *from employees where(department_id,salary) in (select department_id,max(salary)from employees group by department_id)
- **inline view-** From inside select means inline view sub query
  select max(sal) from (select department_id,max(salary) sal from employees group by department_id)
- **scalar Subqueries** - select inside select means Scalar subquery
  If you use select statement inside select statement then it scalar Subqueries

- select first_name,salary,(select avg(salary) from employees)from employees
- **correlated Subqueries-** Outer query refers inner query value or inner query refers outer query values then it is corealeted Subqueries
- select department_name from departments d where department_id in(select department_id from employees where department_id=d.department_id)
- **nested Subqueries-** Where class inside select means nested sub query

  select * from employees where salary in (select max(salary) from employees)

**19. What are Set Operators**

- Union
- Union All
- Minus
- Insersect

**20. What are Psudo Columns**

- Distinct
- Unique
- Rownum
- Rowid
- Sysdate
- Systimestamp
- User
- Uid

**21. What is View and Materialized View**

A view is a stored query. Views are saved in the database server so we do not have to copy them from some other file. We create a view by assigning a name to a query

A materialized view in Oracle is a database object that contains the results of a query. They are local copies of data located remotely, or are used to create summary tables based on aggregations of a table's data.

**22. Refreshing Method in Materialized View**

Materialized views can be refreshed in two ways: fast or complete. A fast refresh requires having a materialized view log on the source tables that keeps track of all changes since the last refresh, so any new refresh only has changed (updated, new, deleted) data applied to the MV

**23. Date Functions in SQL**

| Function | Example | Result | Description |
|---|---|---|---|
| ADD_MONTHS | ADD_MONTHS( DATE '2016-02-29', 1 ) | 31-MAR-16 | Add a number of months (n) to a date and return the same day which is n of months away. |
| CURRENT_DATE | SELECT CURRENT_DATE FROM dual | 06-AUG-2017 19:43:44 | Return the current date and time in the session time zone |

| Function | Example | Result | Description |
|---|---|---|---|
| CURRENT_TIMESTAMP | SELECT CURRENT_TIMESTAMP FROM dual | 06-AUG-17 08.26.52.742000000 PM **-07:00** | Return the current date and time with time zone in the session time zone |
| DBTIMEZONE | SELECT DBTIMEZONE FROM dual; | -07:00 | Get the current database time zone |
| EXTRACT | EXTRACT(YEAR FROM SYSDATE) | 2017 | Extract a value of a date time field e.g., YEAR, MONTH, DAY, … from a date time value. |
| FROM_TZ | FROM_TZ(TIMESTAMP '2017-08-08 08:09:10', '-09:00') | 08-AUG-17 08.09.10.000000000 AM -07:00 | Convert a timestamp and a time zone to a TIMESTAMP WITH TIME ZONE value |
| LAST_DAY | LAST_DAY(DATE '2016-02-01') | 29-FEB-16 | Gets the last day of the month of a specified date. |
| LOCALTIMESTAMP | SELECT LOCALTIMESTAMP FROM dual | 06-AUG-17 08.26.52.742000000 PM | Return a TIMESTAMP value that represents the current date and time in the session time zone. |
| MONTHS_BETWEEN | MONTHS_BETWEEN( DATE '2017-07-01', DATE '2017-01-01' ) | 6 | Return the number of months between two dates. |
| NEW_TIME | NEW_TIME( TO_DATE( '08-07-2017 01:30:45', 'MM-DD-YYYY HH24:MI:SS' ), 'AST', 'PST' ) | 06-AUG-2017 21:30:45 | Convert a date in one time zone to another |
| NEXT_DAY | NEXT_DAY( DATE '2000-01-01', 'SUNDAY' ) | 02-JAN-00 | Get the first weekday that is later than a specified date. |
| ROUND | ROUND(DATE '2017-07-16', 'MM') | 01-AUG-17 | Return a date rounded to a specific unit of measure. |
| SESSIONTIMEZONE | SELECT SESSIONTIMEZONE FROM dual; | -07:00 | Get the session time zone |
| SYSDATE | SYSDATE | 01-AUG-17 | Return the current system date and time of the operating system where the Oracle Database resides. |
| SYSTIMESTAMP | SELECT SYSTIMESTAMP FROM dual; | 01-AUG-17 01.33.57.929000000 PM -07:00 | Return the system date and time that includes fractional seconds and time zone. |

| Function | Example | Result | Description |
|---|---|---|---|
| TO_CHAR | TO_CHAR( DATE'2017-01-01', 'DL' ) | Sunday, January 01, 2017 | Convert a DATE or an INTERVAL value to a character string in a specified format. |
| TO_DATE | TO_DATE( '01 Jan 2017', 'DD MON YYYY' ) | 01-JAN-17 | Convert a date which is in the character string to a DATE value. |
| TRUNC | TRUNC(DATE '2017-07-16', 'MM') | 01-JUL-17 | Return a date truncated to a specific unit of measure. |

24. **OLAP**
    Online Analytical Processing uses complex queries to analyze aggregated historical data from OLTP systems
25. **OLTP**
    Online Transaction Processing captures, Stores & Processes data from Transaction in real time
26. **What is Data Warehousing**
    Data warehousing is the storage of information over time by a business or other organization. New data is periodically added by people in various key departments such as marketing and sales. ... A database is designed to supply real-time information. A data warehouse is designed as an archive of historical information
27. **Delete and Truncate which one Executes First**
    Truncates executes first
28. **Like Operator**

| LIKE Operator | Description |
|---|---|
| WHERE CustomerName LIKE 'a%' | Finds any values that start with "a" |
| WHERE CustomerName LIKE '%a' | Finds any values that end with "a" |
| WHERE CustomerName LIKE '%or%' | Finds any values that have "or" in any position |
| WHERE CustomerName LIKE '_r%' | Finds any values that have "r" in the second position |
| WHERE CustomerName LIKE 'a_%' | Finds any values that start with "a" and are at least 2 characters in length |
| WHERE CustomerName LIKE 'a__%' | Finds any values that start with "a" and are at least 3 characters in length |
| WHERE ContactName LIKE 'a%o' | Finds any values that start with "a" and ends with "o" |

29. **When will you use Having and when you use Where**

- We use WHERE prior to GROUP BY and HAVING after GROUP BY. The Where clause acts as a pre filter where as Having as a post filter.
- The where clause works on row's data, not on aggregated data
  SELECT Name, Score FROM Marks WHERE Score >=40
- The having clause works on aggregated data.
  SELECT Name, SUM(score) AS tot FROM Marks GROUP BY Name HAVING total > 70

**30. What is Stored Procedure**

A stored procedure is a prepared SQL code that you can save, so the code can be reused over and over again

**31. What is Star Schema and Snow Flex Schema**

A **star schema** is a database organizational structure optimized for use in a data warehouse or business intelligence that uses a single large fact table to store transactional or measured data, and one or more smaller dimensional tables that store attributes about the data. It is called a star schema because the fact table sits at the center of the logical diagram, and the small dimensional tables branch off to form the points of the star.

**Snowflake Schema** in data warehouse is a logical arrangement of tables in a multidimensional database such that the ER diagram resembles a snowflake shape. A Snowflake Schema is an extension of a Star Schema, and it adds additional dimensions. The dimension tables are normalized which splits data into additional tables.

# Difference between Questions

**1. Difference b/w Case and Decode**

| S. No | CASE | DECODE |
|---|---|---|
| 1. | It is a statement | It is a function |
| 2. | It can work with logical operators other than ' =' | It performs only equality check |

**2. Difference b/w Rank and Dense_Rank**

| S. No | RANK | DENSE RANK |
|---|---|---|
| 1. | Rank gives you the ranking within your ordered partition. Ties are assigned the same rank, with the next ranking skipped | Dense rank gives you the ranking within your ordered partition, but the ranks are consecutive |
| 2. | Eg[10,20,30,30,40] rank o/p[5,4,2,2,1] | Eg[10,20,30,30,40] rank o/p[4,3,2,2,1] |

NOTE: For nulls, ranking depends on ORDER BY clause.

**3. Difference b/w lead and lag**

| S. No | LEAD | LAG |
|---|---|---|
| 1. | Fetches data from the subsequent row | Fetches data from the previous row |
| 2. | Lead(expression[offset[default_value]]) | Lag(expression[offset[default_value]]) |

**4. Difference b/w View and Materialized View**

| S. No | View | Materialized View |
|---|---|---|
| 1. | It is a virtual table | It is a snap short of database |
| 2. | It stores the query | It stores the data |

**5. Difference b/w Star Schema and Snow flex Schema**

| Star Schema | Snowflake Schema |
|---|---|
| Hierarchies for the dimensions are stored in the dimensional table. | Hierarchies are divided into separate tables. |
| It contains a fact table surrounded by dimension tables. | One fact table surrounded by dimension table which are in turn surrounded by dimension table |
| In a star schema, only single join creates the relationship between the fact table and any dimension tables. | A snowflake schema requires many joins to fetch the data. |
| Simple DB Design. | Very Complex DB Design. |
| Denormalized Data structure and query also run faster. | Normalized Data Structure. |
| High level of Data redundancy | Very low-level data redundancy |
| Single Dimension table contains aggregated data. | Data Split into different Dimension Tables. |

**6. Difference b/w Delete and Truncate and Drop**

| S. No | Delete | Truncate | Drop |
|---|---|---|---|
| 1. | DML command | DDL Command | DDL command |
| 2. | IT s used to remove the row from table | It is used to remove all rows from a table | It removes the table structure |
| 3. | It can be roll back | It cannot be roll back | It cannot be roll back |

**7. Difference b/w Having and Where clause**

| S. No | Having clause | Where clause |
|---|---|---|

| | | |
|---|---|---|
| 1. | Applies only to group as a whole. | Applies to individual role |
| 2. | Only the rows that meet the condtion in the where clause are grouped. | Apply only to the columns that appear group by clause or in an aggregate function |

**8. Difference b/w Normalization and Denormalization**

| S. No | Normalization | Denormalization |
|---|---|---|
| 1. | Normalization is used to remove redundant data from the database and to store non-redundant and consistent data into it. | Denormalization is used to combine multiple table data into one so that it can be queried quickly. |
| 2. | Increase the number of tables | Decrease the number of tables |